

## An introduction to GLib and data types

Do not confuse GLibC with GLib. GLib is a utility library that was developed as a part of GTK+. Although you can use normal C data types for GTK+ programming, it is easier (and more portable) to use GLib data types. For example, many of the functions that operate on *GtkEntry* objects require that the returned buffer not be modified. This means that before you can change the contents of the buffer, it needs to be duplicated. Since the buffer contains input from the user, it is easier (and more secure) to use a GLib function (like `g_strdup()`) rather than trying to allocate a fixed-size character buffer and doing `strcpy()`.

Similarly, we use *GSSlist* (GLib's singly-linked list implementation) and its associated functions (like `g_slist_foreach()`), which loops through all elements in the list and calls a given function on each element) to keep a list of *GtkRadioButton* objects. By using GLib data structures and helper functions, we effectively focus on developing our application, rather than on implementing data structures required by GTK+.

called with the *user\_data* parameter as the pointer you used to connect signals. The handler function for the *Cancel* button's click event calls the helper function *clear\_page()* with 0 as the page number, to clear all fields of Page 0. The function *gtk\_entry\_set\_text()* is used to clear the text entry, by setting the buffer to an empty string. The *OK* button click handler (*on\_lgn\_ok\_btn\_clicked()*) verifies the supplied credentials, and advances to the next page of the notebook widget using *gtk\_notebook\_set\_current\_page()*.

### Search/Submit buttons: edit/view page

The information presented on Page1 is editable by privileged users. Consequently, the search button has to serve two purposes, depending on the context—one of searching the database, and one to submit the edited values to the database. In both cases, reading of user inputs from *GtkEntry* objects is done using *gtk\_entry\_get\_text()*. The return value, which is a constant pointer, needs to be copied to another buffer if you intend to modify it. Our copy function is *g\_strdup()* from GLib.

As an example of user entry verification, use *strtol()* on the employee ID input field. If the input is found to be invalid, set the focus on *empid\_ent* and highlight the text buffer. Check the source to see how this is done. While selecting the region, the *GtkEntry* object is typecast as a *GtkEditable* object; this is possible because *GtkEntry* implements the *GtkEditable* interface.

Then push a message to the status bar, to indicate that the user needs to correct the input. After user inputs are verified, either data is retrieved from the database using *search\_record()*, or modified data is submitted to the database using *submit\_record()*. The status is displayed on the status bar. In order to display the employee ID, the number needs to be converted to a string. Use *g\_strdup\_printf()*, which is a *sprintf* equivalent with the added advantage of allocating the target buffer automatically. Use the utility function *g\_slist\_foreach()* to modify the

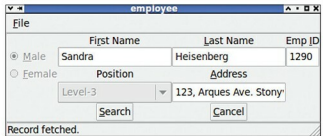


Figure 1: The running application

sensitivity of radio buttons, depending on the privilege level. One of the radio buttons is made active, based on the gender retrieved. We use *gtk\_toggle\_button\_set\_active()*. After setting the employee's position, you can optionally enable editing privileges, and change the button label. As an alternative to greying out fields, make sure that the user cannot click and focus on *GtkEntry* widgets, by setting the *can\_focus* property of these widgets to *FALSE* using *gtk\_widget\_set\_can\_focus()*.

## Compiling

While compiling a GTK+ application, you need to pass the proper arguments to GCC to enable it find the include files. This is true about the linker as well. Since the GTK+ library and include file paths can vary across systems, make use of the *pkg-config* application to resolve file and library locations, and help to pass the correct compiler and linker flags. A typical run of *pkg-config* to find out the compiler options for GTK+-2.0 is:

```
$ pkg-config --cflags gtk+-2.0
-D_REENTRANT -I/usr/include/gtk-2.0 -I/usr/lib/gtk-2.0/
include -I/usr/include/atk-1.0 -I/usr/include/cairo -I/usr/
include/pango-1.0 -I/usr/include/pixman-1 -I/usr/include/
freetype2 -I/usr/include/directfb -I/usr/include/libpng12
-I/usr/include/glib-2.0 -I/usr/lib/glib-2.0/include
```

This tool helps avoid hard-coding of *include* and *library* paths into makefiles. The usage is simple, and you can find examples in our Makefile.

## Running the application

Compiling and running the application is easy:

```
$ make && ./employee
```

Look at the application as shown in Figure 1.

Now that we have completed the application, in the next article in the series, we will look at how to make this application run under Microsoft Windows, with minimal or no changes to the source code. 

By: Om Narasimhan

Om Narasimhan is a Linux Kernel engineer at NetLogic Microsystems, Santa Clara, California. He occasionally dabbles with GTK, perl and bash.